

Cryptography - UE24CS342AA4 Introduction and Classical Ciphers

1. Introduction

Cryptography is the art of private communication in a public environment.

Applications of Cryptography:

- Encryption/Decryption in email
- Whatsapp Encryption
- Sim card Authentication

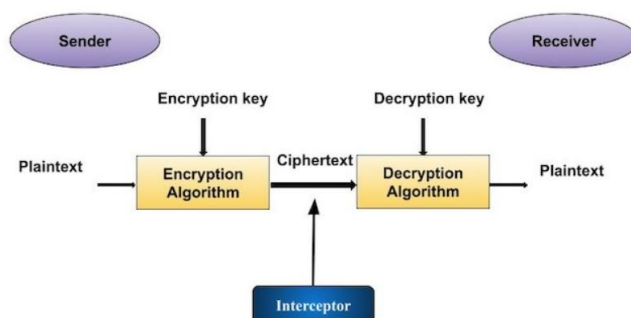
2. Fundamental Terminologies:

- **PlainText** : Human readable
- **CipherText** : Non- Human readable (encrypted message)
- **Encryption** : Conversion of plaintext to ciphertext.
- **Decryption** : Transforming ciphertext back to plaintext.
- **Key** : The secret-numeric value used for encryption.
- **KeySpace**: Total number of all possible key values for a given algorithm.

3. Cryptographic Algorithms

The public knowledge set of rules behind cryptography.

- Modern crypto systems utilize math for this
- Examples: DES (Data Encryption Standard), AES (Advanced Encryption Standard), etc



3.1. How does a Cryptographic Algorithm work?

- A cryptographic algorithm (or cipher) is a special math formula used to **lock (encrypt)** and **unlock (decrypt)** information.
- It always works together with a **key** (a secret word, number, or phrase).
- If you change the key, the same message will turn into a **different secret code (ciphertext)**.
- The **safety** of encrypted data depends on two things:
 1. **How strong the algorithm is**
 2. **How well the key is kept secret**

4. Service of Cryptography – Imp

- **Confidentiality** – Keeps information secret from unauthorized people.
- **Integrity** – Makes sure data is not changed or tampered with.
- **Authentication** – Confirms the identity of the sender/receiver.
- **Non-Repudiation** – Prevents denial of sending or receiving a message.

5. Limitations of Cryptography

- **Availability** – Cryptography cannot guarantee that data or systems are always accessible (e.g., it cannot stop DoS attacks).
- **Key Management** – If keys are lost, stolen, or poorly managed, security breaks completely.
- **Human Factor** – Users may pick weak passwords, share keys, or fall for social engineering, which cryptography cannot prevent.
- **Performance Overhead** – Encryption and decryption take time and resources, which can slow down systems.
- **Legal and Policy Issues** – Strong cryptography may be restricted by laws or misused by attackers for hiding illegal activities.

6. Cryptography law:

Cryptography law (or encryption law) governs the use of cryptography to protect information and personal data. However, since its misuse can hinder law enforcement, governments often regulate or restrict its use.

6.1. Legal Issues with Cryptography

- **Export Control** – Restrictions on sending cryptographic methods or tools from one country to another.
- **Import Control** – Restrictions on using certain cryptographic methods within a country.
- **Patent Issues** – Some cryptographic tools are patented, so their use may require permission or licensing.

Search and Seizure – Laws about when authorities can force someone to decrypt data or reveal encryption keys.

1. Cryptographic Protocols

Definition: Cryptographic protocols are used for application-level data transport security and provide comprehensive security solutions.

Primary Functions of Cryptographic Protocols:

- **Key agreement or establishment:** Securely establishing shared secret keys
- **Entity authentication:** Verifying the identity of communicating parties
- **Symmetric encryption and message authentication:** Protecting data confidentiality and integrity
- **Non-repudiation methods:** Preventing denial of actions or communications
- **Secret sharing methods:** Distributing secrets among multiple parties
- **Secure multi-party computation:** Enabling joint computations without revealing private inputs

Examples of Cryptographic Protocols:

- **IPsec:** Internet Protocol Security for network layer security
- **TLS/SSL:** Transport Layer Security for web communications
- **Kerberos:** Network authentication protocol
- **SSH (Secure Shell):** Secure remote access protocol

User A	Authentication/Key Agreement	Secure Communication	User B
--------	------------------------------	----------------------	--------

2. Cryptographic Primitives

Definition: Well-established, low-level cryptographic algorithms that serve as building blocks for cryptographic protocols in computer security systems.

Classification of Cryptographic Primitives:

Cryptographic Primitives

- └ Secret Key (Symmetric) Cryptography
- └ Public Key (Asymmetric) Cryptography
- └ Unkeyed Primitives (Hash Functions)

3. Types of Cryptography

3.1 Secret Key (Symmetric) Cryptography

- **Key characteristic:** Uses a single key for both encryption and decryption
- **Shared secret:** Same key must be known by both sender and receiver

3.2 Public Key (Asymmetric) Cryptography

- **Key characteristic:** Uses two different but mathematically related keys
- **Key pair:** One key for encryption, another for decryption

3.3 Hash Functions (One-way Cryptography)

- **Key characteristic:** No key required
- **Irreversible:** Plaintext cannot be recovered from the hash output

Symmetric	Key $\leftrightarrow$ [Encrypt/Decrypt]
Asymmetric	Public Key $\rightarrow$ Encrypt Private Key $\rightarrow$ Decrypt
Hash	Data $\rightarrow$ [Hash Function] $\rightarrow$ Digest

4. Symmetric Key Primitives

4.1 Characteristics

- **Single shared key:** Used by both sender and receiver for encryption and decryption
- **Primary purpose:** Provides privacy and confidentiality

- **Security requirements:**
 - Key must be stored securely
 - Secure channel required for key distribution

4.2 Types of Symmetric Primitives

- **Block ciphers:**
 - **AES (Advanced Encryption Standard):** Modern standard block cipher
 - **DES (Data Encryption Standard):** Legacy block cipher
- **Stream ciphers:**
 - **RC4:** Popular stream cipher (now deprecated)
- **Classical ciphers:**
 - **Caesar Cipher:** Historical substitution cipher
- **Supporting components:**
 - **PRNG (Pseudo-Random Number Generators):** For key and IV generation



5. Public Key Primitives

5.1 Key Structure

- **Key pair:** Each participant possesses two mathematically related keys
 - **Private key:** Kept secret by the owner
 - **Public key:** Shared openly with everyone

5.2 Encryption/Decryption Modes

- **Mode 1:** Encrypt with public key → Decrypt with private key (for confidentiality)
- **Mode 2:** Encrypt with private key → Decrypt with public key (for authentication/signatures)

5.3 Main Components

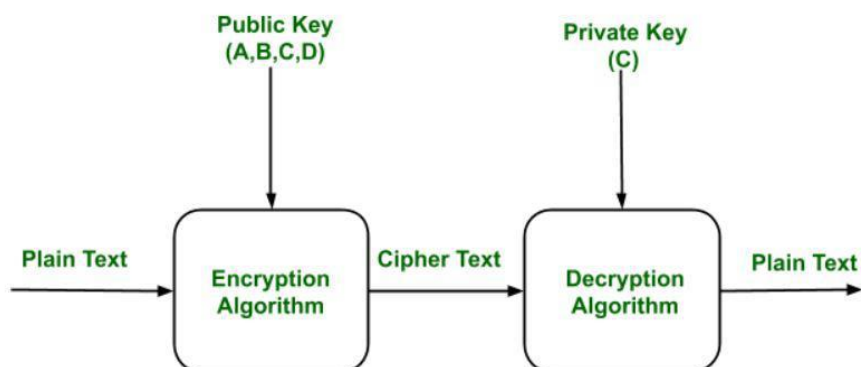
- **Plaintext:** Original message
- **Encryption algorithm:** Mathematical transformation for encryption
- **Private key:** Secret key known only to owner
- **Public key:** Publicly available key
- **Decryption algorithm:** Mathematical transformation for decryption
- **Ciphertext:** Encrypted message

5.4 Important Operational Notes

- **Key generation:** Receiver creates the key pair
- **Process initiation:** Encryption process starts with the receiver, not sender
- **Key distribution:** Public key is distributed openly, private key remains secret
- **Sender verification:** Achieved through digital signatures
- **Primary use case:** Often used to securely exchange symmetric keys

5.5 Examples of Public Key Systems

- **Diffie-Hellman:** Key exchange protocol
- **ECC (Elliptic Curve Cryptography):** Efficient public key system
- **RSA:** Widely used public key algorithm
- **ElGamal:** Public key cryptosystem
- **PKI (Public Key Infrastructure):** Complete framework for managing public keys



6. Unkeyed Primitives (Hash Functions)

6.1 Characteristics

- **Also known as:** One-way cryptography
- **Mathematical nature:** Irreversible transformation of information
- **Output:** Produces a digital fingerprint of the input
- **Key requirement:** No key needed
- **Irreversibility:** Plaintext cannot be recovered from hash output

6.2 Primary Uses

- **Message integrity:** Detecting unauthorized changes to data
- **Password storage:** Storing password hashes instead of actual passwords
- **Digital certificates:** Creating unique identifiers
- **One-way permutations:** Mathematical transformations

6.3 Examples of Hash Functions

- **MD5:** 128-bit hash (now considered weak)
- **SHA-1:** 160-bit hash (deprecated for cryptographic use)
- **SHA-2 family:** SHA-224, SHA-256, SHA-384, SHA-512
- **Whirlpool:** 512-bit hash function
- **Blake:** Modern hash function family

7. Strength of Cryptosystems: The Importance of Keyspace

- Security of a cryptosystem depends heavily on **keyspace size**.
- **Exhaustive key search:** attacker tries every possible key until correct one is found.
- On average, **50% of keys** must be tested before success.
- **Key length impact:** each extra bit doubles the keyspace, making attacks exponentially harder.
- Example: A 100-bit key $\rightarrow 2^{100}$ keys $\rightarrow$ infeasible to break even with massive computing power.
- **Larger keyspace = stronger resistance** against brute-force attacks.

8. Cryptanalysis and Attack Methods

8.1 Cryptanalysis

- **Definition:** Breaking cryptographic systems without knowing the key.

- **Focus:** Exploits the **mathematical structure** of algorithms.
- Methods may use:
 - Algorithm analysis
 - Plaintext patterns or characteristics
 - Known plaintext–ciphertext pairs
 - Statistical analysis (e.g., frequency analysis)
- Goal: Find **shortcuts** to recover key or plaintext faster than brute force.

8.2 Brute-Force Attack

- **Definition:** Systematically testing all possible keys until correct one is found.
 - **Characteristics:**
 - Exhaustive search of keyspace
 - Success guaranteed (on average after 50% of keyspace)
 - Time depends on computational power
 - Universally applicable to any cipher
 - **Resistance:**
 - Based on **large keyspaces**
 - Modern systems ensure brute force is computationally infeasible
 - A cipher is considered **secure** if brute force is the best possible attack
-

1. Introduction to Classical Cryptography

Classical cryptography refers to traditional encryption methods used before the advent of modern computer-based cryptographic systems. These methods were primarily used for centuries to secure military, diplomatic, and personal communications.

Key Concepts:

- **Plaintext:** The original, readable message
- **Ciphertext:** The encrypted, unreadable message
- **Key:** The secret information used to encrypt/decrypt
- **Encryption:** Converting plaintext to ciphertext
- **Decryption:** Converting ciphertext back to plaintext

2. Types of Classical Coding

2.1 Letter Coding

- Uses alphabetic characters as the basis for encryption
- Letters are substituted or rearranged according to specific rules
- Example: $A \rightarrow D$, $B \rightarrow E$, $C \rightarrow F$ (Caesar cipher with shift 3)

2.2 Number Coding

- Converts letters to numbers before applying encryption
- Often uses $A=0$, $B=1$, $C=2$, ... $Z=25$ or $A=1$, $B=2$, ... $Z=26$
- Example: "HELLO" $\rightarrow$ [7, 4, 11, 11, 14] (using $A=0$ system)

2.3 Mixed Coding

- Combines both letters and numbers in the encryption process
- May use numeric keys with alphabetic substitution
- More complex but potentially more secure

3. Cryptosystem Requirements

A practical cryptosystem must satisfy these essential criteria:

3.1 Efficiency Requirements

- **Encryption function (e_k):** Must be efficiently computable
- **Decryption function (d_k):** Must be efficiently computable
- Both operations should be fast enough for practical use

3.2 Security Requirements

- **Key Security:** An opponent seeing ciphertext y should be unable to determine the key k
- **Message Security:** An opponent should be unable to determine the plaintext x from ciphertext y
- The system should resist cryptanalysis attempts

4. Classification of Classical Ciphers

Classical Ciphers

- |— Substitution Ciphers
 - | |— Monoalphabetic
 - | |— Polyalphabetic
 - | |— Polygraphic
- |— Transposition Ciphers
- |— Combined Ciphers

5. Substitution Ciphers

Definition: Encrypt plaintext by replacing each letter or symbol with a different symbol according to the key.

5.1 Monoalphabetic Substitution Cipher

Characteristics:

- Uses a single, fixed substitution alphabet throughout the entire message
- Same plaintext letter always produces the same ciphertext letter
- Vulnerable to frequency analysis

5.1.1 Caesar Cipher

Description: The simplest monoalphabetic substitution cipher where each letter is shifted by a fixed number of positions in the alphabet.

Formula: $C = (P + k) \bmod 26$ (for encryption) **Decryption:** $P = (C - k) \bmod 26$

Example:

- **Plaintext:** "HELLO WORLD"
- **Key:** 3 (shift by 3 positions)
- **Encryption process:**
 - $H \rightarrow K, E \rightarrow H, L \rightarrow O, L \rightarrow O, O \rightarrow R$
 - $W \rightarrow Z, O \rightarrow R, R \rightarrow U, L \rightarrow O, D \rightarrow G$
- **Ciphertext:** "KHOOR ZRUOG"

Alphabet Mapping (Shift 3):

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
D E F G H I J K L M N O P Q R S T U V W X Y Z A B C

5.2 Polyalphabetic Substitution Cipher

Characteristics:

- Uses multiple substitution alphabets
- The cipher alphabet changes at different positions during encryption
- More secure than monoalphabetic ciphers

5.2.1 Playfair Cipher

Description: Encrypts pairs of letters (digraphs) using a 5×5 grid of letters.

Key Setup:

1. Create a 5×5 grid using a keyword (remove duplicates, combine I/J)
2. Fill remaining spaces with unused alphabet letters

Example with Key "ENCRYPT":

E N C R Y
P T A B D
F G H I/J K
L M O Q S
U V W X Z

Encryption Rules:

1. **Repeated letters:** Insert filler (X) between them
2. **Same row:** Replace each letter with the one to its right (wrap around)
3. **Same column:** Replace each letter with the one below (wrap around)
4. **Rectangle:** Replace each letter with the one in its row but in the other letter's column

Complete Example:

- **Plaintext:** "TELL HIM ABOUT ME"
- **Step 1 - Handle repeated letters:** "TE LX LH IM AB OU TM EX"
- **Step 2 - Apply rules:**
 - TE → PN (rectangle rule)
 - LX → QU (rectangle rule)
 - LH → OF (rectangle rule)
 - IM → GQ (rectangle rule)
 - AB → BD (same row, shift right)
 - OU → LW (rectangle rule)
 - TM → GV (rectangle rule)
 - EX → RU (rectangle rule)
- **Ciphertext:** "PNQUOFGQBDLWGV RU"

5.2.2 Vigenère Cipher

Description: Uses a repeating keyword to determine the shift for each letter, essentially multiple Caesar ciphers.

Formula: $C_i = (P_i + K_i) \bmod 26$

Example 1 - Numeric Key:

- **Plaintext:** "I ATTACK"
- **Key:** [2, 3, 4, 2, 3, 4, 2, 3] (repeated pattern)
- **Encryption:**
 - I (8) + 2 = K (10)
 - A (0) + 3 = D (3)
 - T (19) + 4 = X (23)
 - T (19) + 2 = V (21)
 - A (0) + 3 = D (3)
 - C (2) + 4 = G (6)
 - K (10) + 2 = M (12)
- **Ciphertext:** "KDXVDGM"

Example 2 - Keyword:

- **Plaintext:** "I ATTACK"
- **Keyword:** "EXAM" (repeated as needed)
- **Key arrangement:** E X A M E X A M
- **Encryption:**

- $I + E = N$
- $A + X = Y$
- $T + A = R$
- $T + M = G$
- $A + E = F$
- $C + X = A$
- $K + A = L$

- **Ciphertext:** "NYRGFAL"

5.3 Polygraphic Substitution Cipher

Characteristics:

- Works on multiple letters simultaneously (blocks of letters)
- More complex mathematical operations
- Higher security than single-letter substitutions

5.3.1 Hill Cipher

Description: Uses matrix multiplication for encryption and decryption.

Mathematical Foundation:

- Based on the concept: $M \times M^{-1} = I$ (where I is the identity matrix)
- Requires the key matrix to have a multiplicative inverse modulo 26

Example Setup: Key Matrix M:

$$M = \begin{bmatrix} 3 & 0 & 2 \\ 2 & 0 & -2 \\ 0 & 1 & 1 \end{bmatrix}$$

Encryption Process:

1. Convert plaintext to numerical vectors ($A=0, B=1, \dots, Z=25$)
2. Multiply each vector by the key matrix
3. Take results modulo 26
4. Convert back to letters

Example:

- **Plaintext:** "CAT"

- **Numerical:** [2, 0, 19]
- **Matrix multiplication:** [2, 0, 19] $\times$ M mod 26
- **Result:** Calculate each component and convert back to letters

Decryption: Use the inverse matrix M^{-1} for decryption

6. Classical Transposition Ciphers

Definition: A method of encryption where the positions of plaintext units (characters or groups of characters) are shifted according to a regular system, creating a ciphertext that is a permutation of the plaintext.

Key Principle: Characters are shuffled in a regular pattern rather than being substituted with different characters.

6.1 Rail Fence Cipher (Zigzag Cipher)

Description: A transposition cipher that rearranges message letters using alternating lines in a zigzag pattern.

Algorithm:

1. Write the message on alternate lines across the page
2. Read off each line in turn to form the ciphertext

Example with 3 Rails:

- **Plaintext:** "HELLO WORLD"
- **Writing pattern:**

```
H L O L
E L W R D
L O R
```

- **Reading by rails:** "HLOL" + "ELWRD" + "LOR"
- **Ciphertext:** "HLOLEELWRDLOR"

Example with 2 Rails (simpler version):

- **Plaintext:** "WE ARE DISCOVERED"

- **Rail 1:** W _ A _ E _ I _ C _ V _ R _ D
- **Rail 2:** _ E _ R _ D _ S _ O _ E _ E _
- **Ciphertext:** "WAEICVRD" + "ERDSOE EEE" = "WAEICVRDERDSOE EEE"

6.2 Columnar Transposition Cipher

Description: Message is written in rows of fixed length, then read column by column in a scrambled order determined by a keyword.

Algorithm Steps:

1. Choose a keyword to determine column order
2. Write plaintext in rows under the keyword
3. Number columns based on alphabetical order of keyword letters
4. Read columns in numerical order

Complete Example:

- **Plaintext:** "WE ARE DISCOVERED FLEE AT ONCE"
- **Keyword:** "ZEBRAS" (length 6)
- **Column ordering:** Z=6, E=3, B=2, R=4, A=1, S=5
- **Alphabetical order:** A(1), B(2), E(3), R(4), S(5), Z(6)

Step 1 - Write in rows:

```
Z E B R A S
6 3 2 4 1 5
-----
W E   A R E
  A R E   D
I S C O V E
R E D   F L
E E   A T
O N C E
```

Step 2 - Read by column numbers:

- Column 1 (A): R, E, V, F, T
- Column 2 (B): (empty), R, C, D, (empty), C
- Column 3 (E): E, A, S, E, E, N
- Column 4 (R): A, E, O, (empty), A, E
- Column 5 (S): E, D, E, L, (empty), (empty)

- Column 6 (Z): W, (empty), I, R, E, O

Step 3 - Handle nulls: Fill empty spaces with nulls (X)

- **Ciphertext:** "REVFT XRCDXC EASEN AEXAE EDLXX WXIREO"

6.3 Columnar Transposition Decryption

Decryption Process:

1. **Calculate column lengths:** Divide message length by key length
2. **Determine column distribution:** Account for remainder characters
3. **Write ciphertext in columns:** Following the key order
4. **Reorder columns:** Based on alphabetical order of keyword
5. **Read row by row:** To recover plaintext

Example Decryption:

- **Ciphertext length:** 30 characters
- **Key length:** 6
- **Column length:** $30 \div 6 = 5$ characters per column
- Reconstruct the grid and read horizontally

6.4 Double Transposition Cipher

Description: Two successive applications of columnar transposition, potentially using different keys for enhanced security.

Complete Example:

- **Plaintext:** "WE ARE DISCOVERED FLEE AT ONCE"
- **Key 1:** "ZEBRAS"
- **Key 2:** "HELP"

First Transposition (Key: ZEBRAS):

- Apply columnar transposition as shown above
- **Intermediate result:** "EVLNE ACDTK ESEAQ ROFOJ DEECU WIREE"

Second Transposition (Key: HELP):

- **Column order for HELP:** H=2, E=1, L=3, P=4
- Apply columnar transposition to intermediate result
- **Final Ciphertext:** "EVAKAFEWEETEODUELCEQOEIKNDSRJCRG"

Security Advantage: Double transposition significantly increases cryptanalytic difficulty compared to single transposition.

7. Security Analysis

7.1 Vulnerabilities of Classical Ciphers

Classical ciphers (Caesar, Monoalphabetic, Vigenère, Transposition, etc.) were simple to implement but had inherent weaknesses that made them insecure once systematic cryptanalysis developed.

1. Frequency Analysis

- Languages (e.g., English) have uneven letter distributions.
- Example: In English, **E, T, A, O, I, N** are the most frequent letters.
- Monoalphabetic substitution preserves these patterns, making it easy to guess substitutions.

2. Pattern Recognition

- Repeated words or letter groups in the plaintext produce **repeated ciphertext patterns**.
- Example: The word “HELLO” may always map to the same ciphertext sequence.
- Attackers look for repeated ciphertext chunks to deduce possible words.

3. Limited Key Space

- Many classical ciphers have very small key spaces, making brute-force attacks feasible.
- Example: Caesar cipher has only **25 possible shifts**.

- Even more complex polyalphabetic ciphers have manageable key lengths compared to modern cryptography.

4. Known Plaintext Attacks

- If an attacker has access to both a ciphertext and part of the corresponding plaintext, the key can often be deduced.
- Example: In substitution ciphers, knowing just a few mappings can quickly expose the rest.

7.2 Cryptanalysis Methods

1. Brute Force

- Attackers try every possible key until the correct plaintext appears.
- Works quickly on ciphers with small key spaces (e.g., Caesar, simple substitution).

2. Frequency Analysis

- Based on the fact that natural language letters are not evenly distributed.
- Example: In English, “E” $\approx$ 13%, “T” $\approx$ 9%, while “Z” $<$ 0.1%.
- By comparing ciphertext frequencies with expected distributions, likely substitutions can be identified.

3. Kasiski Examination (1843)

- Designed to break the **Vigenère cipher**.
- Steps:
 - Find repeated sequences of letters in ciphertext.
 - Measure the distance (number of characters apart).

- Compute greatest common divisors (GCDs) of these distances → likely key length.
 - Once key length is known, treat it as several Caesar ciphers, and break each separately with frequency analysis.
-

1. Mathematics in Cryptographic Primitives

1.1 Mathematics by Primitive Type

Unkeyed Primitives (Hash Functions)

- **Primary operations:** Bit manipulation and swapping
- **Complexity:** Relatively simple mathematical operations

Symmetric Key Cryptography

- **Mathematical foundation:** Binary Galois Fields $GF(2^k)$
- **Operations:** Permutations, matrix multiplication
- **Algebraic structures:** Finite field arithmetic

Public Key Cryptography

- **Mathematical foundation:** Number theory
- **Core concepts:** Used in essentially all public key systems including ECC
- **Advanced mathematics:** Requires deep understanding of mathematical concepts

1.2 Mathematics in Cryptanalysis

- **Probability and statistics:** For analyzing cipher strength and randomness
- **Random oracle models:** Theoretical frameworks for security proofs
- **Number theoretical algorithms:**
 - Primality testing
 - Integer factorization
- **Discrete logarithms:**
 - Cyclic group theory
 - Index calculus methods

- **Elliptic curve mathematics:**
 - Point counting on elliptic curves
 - Theory of elliptic curves
-

Probability Basics

Probability measures how likely an event is to happen.

Formula:

$$P(E) = \frac{\text{Favourable Outcomes}}{\text{Total Outcomes}}$$

- **Event** – An outcome or a set of outcomes from an experiment (e.g., getting a head in a coin toss).
- **Sample Space (S)** – The set of all possible outcomes (e.g., {Head, Tail}).
- **Random Experiment** – An experiment where the outcome cannot be predicted with certainty (e.g., rolling a die).

Example: Tossing a coin → Probability of head:

$$P(\text{Head}) = \frac{1}{2}$$

Conditional Probability

- Probability of an event **A** happening given that **B** has already happened.
- Formula:

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

Practice Question:

Relative frequencies in English text:

- a: 0.08
- e: 0.13
- i: 0.07
- o: 0.07
- u: 0.03

Q1: $P(X \text{ is a vowel}) = 0.08 + 0.13 + 0.07 + 0.07 + 0.03 = 0.38$

Q2: $P(X=e | X \text{ is a vowel}) = \frac{0.13}{0.38} = 0.34$

$$Q3: P(X \text{ is a vowel} \mid X \neq a) = \frac{0.13+0.07+0.07+0.03}{1-0.08}$$

Why Probability Matters in Cryptography

- **Random Number Generation** – Keys and nonces must be unpredictable.
- **Probabilistic Encryption** – Adds randomness for stronger security.
- **Error Detection in Quantum Crypto** – Uses probability to catch eavesdropping.
- **Security Proofs** – Based on probabilistic adversaries and reductions.
- **Hash Functions** – Birthday paradox shows why collisions matter.
- **Zero-Knowledge Proofs** – Built on probabilistic interactions.
- **Statistical Tests** – Check randomness and entropy.

Perfect Secrecy

- An encryption scheme (Gen, Enc, Dec) is **perfectly secret** if seeing the ciphertext gives **no extra information** about the plaintext.

$$\Pr[M=m \mid C=c] = \Pr[M=m]$$

This means: **Even after seeing the ciphertext, the chance of guessing the original message remains the same.**

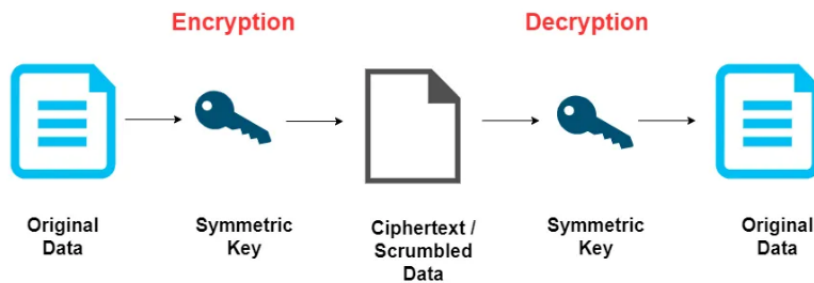
Modern Cryptography

Modern cryptography uses mathematics and computational hardness to secure communication and information.

It is not just about hiding text but about building trust, authenticity, and secure digital systems.

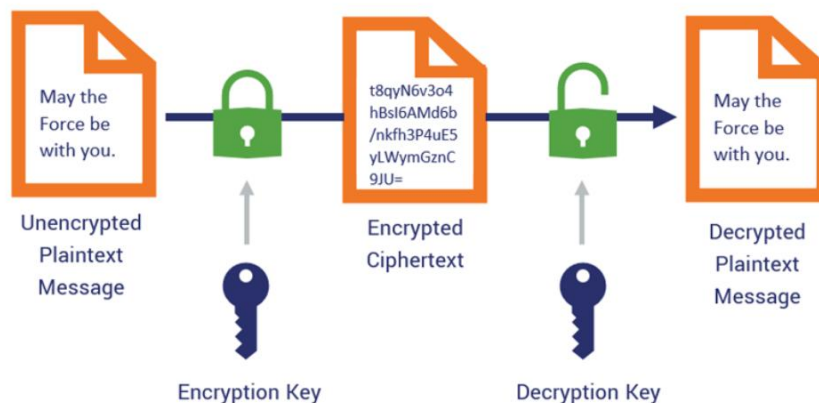
3. Symmetric Key Cryptography

- One shared key for both encryption and decryption.
- Efficient and widely used for bulk data.
- Examples: AES, DES, Blowfish.



4. Asymmetric Key Cryptography

- Uses a pair of keys: public (shared openly) and private (kept secret).
- Public key encrypts, private key decrypts.
- Examples: RSA, Diffie-Hellman.



Classical vs Modern

Classical Cryptography

Works on alphabets

Shared secret among all

Security depends on secrecy of method

Simple ciphers (Caesar, Vigenère)

Modern Cryptography

Works on bits

Keys unique to sender/receiver

Security depends on mathematical strength

Advanced algorithms (AES, RSA, ECC)

Everyday Applications

- Password protection
- Secure WiFi and internet protocols
- Online banking and digital payments

- Software authentication
- Cryptocurrencies and blockchain

Principles of Design

1. Define clear security objectives.
2. Minimize assumptions about the system.
3. Base security on solid mathematical proofs.

Cipher Formalism

A cipher is represented as:

(P,C,K,E,D) where:

- P: plaintexts
- C: ciphertexts
- K: keys
- E: encryption function
- D: decryption function

Correctness condition:

$$D_k(E_k(x)) = x$$

Example (Caesar shift):

$$E_k(x) = (x + k) \bmod 26$$

$$D_k(x) = (y - k) \bmod 26$$

Kerckhoffs's Principle

"A cryptographic system should be secure even if everything about the system, except the key, is public knowledge."

This ensures transparency and allows algorithms to be openly tested without losing security.

Features of an Ideal Cryptosystem

- Security guaranteed under all circumstances.
 - Simple to use and manage.
 - Flexible in application.
 - Resistant even if ciphertext is stolen.
-

Shannon's Equation and Perfect Secrecy

Encryption Scheme Basics

An encryption scheme is defined by three algorithms:

- **Gen**: probabilistically generates a key k from the key space K .
- **Enc**: takes $k \in K$ and $m \in M$ (message space) and produces a ciphertext $c \in C$.
- **Dec**: takes $k \in K$ and $c \in C$ and outputs the original message $m \in M$.

Correctness requires:

$$Dec_k(Enc_k(m)) = m$$

Example (One-Time Pad Intuition)

- Message space M : all possible 20-letter messages Alice could send.
- Key space K : all possible 20-length random shifts (1–26) generated by Gen.
- Ciphertext space C : all possible results after encryption.

If $|M|=|K|=|C|$, then every plaintext is equally likely under encryption.

This balance of sizes is what enables **perfect secrecy**.

What Perfect Secrecy Means

An encryption scheme is perfectly secret if:

- The ciphertext leaks no additional information about the plaintext.
- The probability distribution of possible plaintexts does not change after seeing the ciphertext.

Formally for every distribution over M , for all $m \in M$ and $c \in C$ with $\Pr[C=c] > 0$

$$\Pr[M=m|C=c] = \Pr[M=m]$$

This condition ensures that even an attacker with unlimited computational power learns nothing new about the message from the ciphertext.

Security Intuition

- It must be impossible to deduce the key from ciphertext.
- It must be impossible to recover the plaintext or even any part of it.
- The ciphertext should look like pure randomness.

One-Time Pad (OTP)

Basic Idea

- Extend Vigenère cipher → use a **key as long as the plaintext**.
- Encryption/Decryption done with **XOR ($\oplus$)** operation.
- No repeats, no frequency analysis possible → theoretically unbreakable if rules are followed.

Encryption/Decryption

- Plaintext m , Key k , Ciphertext c
- $Enc_k(m) = m \oplus k$
- $Dec_k(c) = c \oplus k$
- Correctness: $k \oplus (m \oplus k) = m$

Why it is Perfectly Secure

- Every ciphertext could correspond to *any* plaintext of the same length.
- If the key is random, attacker learns nothing from the ciphertext.

Strict Rules for OTP

1. **Key length = Message length.**
2. **Key must be truly random.**
3. **Key must be used only once** (reuse → Two-Time Pad → insecure).
4. **Only 2 copies of the key** (sender & receiver).
5. **Key destroyed after use.**

What if Rules are Broken?

- Key from a book/not random → Not OTP anymore → can be broken.
- Key reused → patterns leak → insecure.

Why Not Used Today?

- Requires very long random keys.
- Hard to generate true randomness.
- Key exchange & storage is impractical at scale.

Why Still Important?

- Serves as the **gold standard of secrecy** (perfect secrecy).
- Foundation for modern cryptographic security proofs.

Example:

Encrypt the letter "I" using OTP.

Step 1: Convert to Binary

- ASCII of "I" = 73 → Binary = 01001001

Step 2: Choose a Random Key

- Random 8-bit key: 10110101

Step 3: XOR Encryption

- XOR each bit of plaintext with key:

Plaintext (P)	Key (K)	Ciphertext
0	1	1
1	0	1
0	1	1
0	1	1
1	0	1
0	1	1
0	0	0
1	1	0

Ciphertext = 11111100

Step 4: Decrypt

- XOR ciphertext with the same key: 11111100 XOR 10110101 = 01001001
- Binary 01001001 → ASCII 73 → Letter "I"

Key points:

- The key must be **truly random** and **used only once**.
- OTP gives **perfect secrecy**.

- XOR operation is reversible, so encryption = decryption with the same key.